



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y
DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica Industrial y Automática

PROYECTO DE INFORMÁTICA INDUSTRIAL

**DESARROLLO DEL VIDEOJUEGO ARCHON
MEDIANTE PROGRAMACIÓN
ORIENTADA A OBJETOS EN C++**

RANCHO

Álvaro Pérez Ruano

Julián Osiniri Juárez

Miguel Vázquez Aguado

Jaime Ladero Blanco

David San Román González

Madrid, Mayo de 2026



ÍNDICE

1. INTRODUCCIÓN	3
1.1. Contexto	3
1.2. Origen de "Rancho"	4
1.3. Objetivos del proyecto.....	5
1.3.1. Desacople lógico-gráfico	5
1.3.2. Aplicación de jerarquías polimórficas	5
1.3.3. Implementación de mecánicas duales.....	5
1.3.4. Gestión robusta de memoria	5
1.3.5. Metodología de desarrollo colaborativo	6
1.3.6. Requisitos opcionales	6
2. MANUAL DE INSTRUCCIONES	6
2.1. Requisitos del sistema y dependencias	6
2.2. Guía de inicio	6
2.3. Controles.....	6
2.4. Reglas del juego	6
2.4.1. Condiciones de Victoria.....	6
2.4.2. El Tablero.....	7
2.4.3. Desplazamiento de los Animales	7
2.4.4. La Arena	8
2.4.5. Habilidades del Granjero	8
3. ARQUITECTURA DEL SOFTWARE (POO).....	9
3.1. Desacople entre lógica y motor gráfico	9
3.1.1. Limpieza del Modelo.....	9
3.1.2. Encapsulación y const-correctness	9
3.2. Estructura de clases	10

4. LÓGICA DE JUEGO.....	10
4.1. Fase estratégica (Tablero).....	10
4.1.1. Validación de movimientos	10
4.1.2. Habilidades especiales.....	13
4.2. Fase de acción (Arena).....	16
4.3. Las piezas (Animales)	26
5. METODOLOGÍA Y CONTROL DE VERSIONES.....	34
5.1. Flujo de trabajo (Gitflow).....	35
5.2. Integración y revisión de código	35
5.3. Estructura del repositorio	37
5.4. Guía de estilo y convenciones de código.....	38
5.4.1. Nomenclatura.....	38
5.4.2. Formato y sintaxis	39
5.4.3. Arquitectura y POO	39
6. ELEMENTOS ORIGINALES Y OPCIONALES	40
6.1. Personalización temática	40
6.1.1. Gráficos y animaciones	40
6.1.2. Música y efectos de sonido.....	41
6.2. Funcionalidades extra.....	41
7. CONCLUSIONES	41

1. INTRODUCCIÓN

1.1. Contexto

Este proyecto se enmarca en la asignatura de Informática Industrial, como trabajo obligatorio a realizar en el tercer curso del Grado en Ingeniería Electrónica Industrial y Automática ofrecido por la Escuela Técnica Superior de Ingeniería y Diseño Industrial (ETSIDI) de la Universidad Politécnica de Madrid.

El proyecto consiste en el diseño y desarrollo en C++ de un videojuego estructurado bajo el paradigma de la Programación Orientada a Objetos (POO). Tomando como referencia las mecánicas fundamentales del clásico *Archon: The Light and the Dark* (1983) y adaptándolas al entorno moderno de Visual Studio 2026 junto a OpenGL (mediante *freeglut*) y la librería *ETSIDIlib* para el renderizado y gestión de recursos multimedia de realización propia.

El código del videojuego se ha concebido desde un inicio respetando la normativa del proyecto, garantizando un fuerte desacoplamiento entre el motor gráfico y las estructuras lógicas, aplicando patrones de diseño y gestionando los recursos dinámicos mediante el patrón RAI (Resource Acquisition Is Initialization) para asegurar la robustez y evitar fugas de memoria.



Figura 1. Archon: The Light and the Dark (1983)

1.2. Origen de "Rancho"

Rancho nace como una reinterpretación original del videojuego clásico *Archon*. Tanto es así que, cambiando un par de letras en forma de anagrama, se puede pasar del nombre original al de este proyecto.



Figura 2. Menú principal de *Rancho*.

Archon: The Light and the Dark está ambientado en un universo de fantasía heroica donde criaturas mitológicas, hechiceros y monstruos se enfrentan en una eterna batalla entre las fuerzas de la luz y la oscuridad. En *Rancho*, esta épica confrontación se traslada a un entorno mucho más terrenal, rústico y cómico: una granja.

La solemne dualidad entre el bien y el mal se sustituye por la feroz rivalidad territorial entre los distintos animales de corral. El tablero de casillas representa las parcelas de la granja que cambian de estado, y la arena de combate simula los caóticos encontronazos que ocurren cuando dos especies se disputan la misma parcela.

Esta temática no solo aporta un tono humorístico y desenfadado, sino que justifica a nivel de diseño de gameplay una rica variedad de estadísticas, habilidades y tipos de ataque. Las diferencias anatómicas y de comportamiento de cada animal encajan a la perfección con la necesidad técnica de diseñar una jerarquía polimórfica variada, permitiendo diferenciar, por ejemplo, los ataques de embestida de una cabra con los lanzamientos de huevo de una gallina.

1.3. Objetivos del proyecto

El objetivo principal de este trabajo es asentar y demostrar de forma práctica los conocimientos sobre Programación Orientada a Objetos adquiridos durante el curso, materializándolos en un programa estructurado, eficiente y libre de errores. Para alcanzar este propósito general, se establecen los siguientes objetivos alineados con la normativa del proyecto.

1.3.1. Desacople lógico-gráfico

Garantizar una separación estricta entre la lógica (estructuras de datos, validación de reglas, estados de colisión) y el motor de renderizado gráfico, de modo que los métodos de las clases del juego no dependan directamente de las librerías gráficas.

1.3.2. Aplicación de jerarquías polimórficas

Modelar las entidades del juego mediante una clase base abstracta Animal que delegue el comportamiento específico a sus clases derivadas mediante enlace dinámico y métodos virtuales, maximizando la escalabilidad y reutilización del código.

1.3.3. Implementación de mecánicas duales

Desarrollar y coordinar dos lógicas de juego independientes pero interconectadas: una fase estratégica por turnos apoyada en matrices (Tablero), y una fase de acción en tiempo real (Arena) que incluya algoritmos de detección de colisiones (AABB) y cinemática en un espacio continuo 2D.

1.3.4. Gestión robusta de memoria

Aplicar rigurosamente el patrón RAII en la construcción y destrucción de pantallas, entidades y recursos multimedia, garantizando la ausencia de fugas de memoria durante el ciclo de vida del programa.

1.3.5. Metodología de desarrollo colaborativo

Emplear un sistema de control de versiones mediante el flujo de trabajo Gitflow. Esto incluye la gestión de temas mediante issues, el aislamiento de funcionalidades en ramas feature/, y la integración de código hacia la rama develop a través de Pull Requests, simulando un entorno de producción profesional.

1.3.6. Requisitos opcionales

Incluir algunos opcionales

2. MANUAL DE INSTRUCCIONES

2.1. Requisitos del sistema y dependencias

Instrucciones detalladas para configurar Visual Studio 2026, freeglut y ETSIDilib (también cómo enlazar dependencias no caben en el .zip).

2.2. Guía de inicio

Pasos para compilar el proyecto.

2.3. Controles

División de inputs (WASD y Flechas), uso del menú/HUD.

2.4. Reglas del juego

La dinámica de "Rancho" combina la planificación táctica por turnos con la acción en tiempo real, basándose en un conjunto de reglas estrictas.

2.4.1. Condiciones de Victoria

Para alzarse con la victoria del rancho, un jugador debe cumplir con uno de los siguientes tres objetivos:

- Controlar los cinco "puntos de poder" (parcelas resaltadas de color verde claro) del tablero simultáneamente.
- Eliminar por completo a todos los animales del oponente.
- Dejar al bando rival con una única pieza y que esta se encuentre bajo el efecto de la habilidad "Atrapar".

2.4.2. El Tablero

La partida se desarrolla sobre un tablero de 9x9 casillas que representa el terreno de la granja. Cada bando comienza la partida con 18 piezas, divididas en 8 especies distintas, que se disponen inicialmente en las dos columnas laterales del tablero.

El escenario presenta dos mecánicas fundamentales:

- Parcelas oscilantes: un tercio de las casillas del tablero cambia cíclicamente su estado (representando el día/noche o afinidad territorial). Combatir en una casilla cuyo estado coincida con la afinidad de la facción otorga ventaja táctica.
- Puntos de poder: existen cinco casillas clave (una en el centro del tablero y cuatro en los centros de cada borde). Los animales que ocupan estas posiciones regeneran su salud más rápido y son inmunes a las habilidades especiales del rival.

2.4.3. Desplazamiento de los Animales

Durante su turno, el jugador puede mover una de sus piezas. La movilidad depende de la anatomía y tipo del animal, clasificándose en tres categorías:

- Terrestres: deben rodear los obstáculos, no pudiendo moverse en diagonal ni atravesar casillas ocupadas.
- Voladores: tienen movimiento libre y pueden sortear obstáculos en su trayectoria, aunque no pueden finalizar su desplazamiento en una parcela ya ocupada por un aliado.
- Tractor: capacidad de moverse a cualquier parcela válida del tablero. El granjero puede coger el Tractor para “teletransportarse”.

2.4.4. La Arena

Cuando un animal intenta moverse a una parcela ocupada por un animal rival, no se produce una captura automática como en el ajedrez. En su lugar, el juego pasa a una arena de combate donde ambos jugadores luchan en tiempo real. El resultado del combate está influenciado por la habilidad de los jugadores y por atributos únicos de cada animal:

- Fuerza de impacto y tipo de arma (ataque cuerpo a cuerpo o lanzamiento de proyectiles).
- Velocidad de movimiento y de ataque.
- Tiempo de recarga (intervalo necesario entre ataques consecutivos).
- Salud máxima.

Al finalizar el combate, el animal perdedor es eliminado (pudiendo darse la eliminación mutua) y el ganador reclama la parcela disputada.

2.4.5. Habilidades del Granjero

El Granjero es la pieza central de cada bando (equivalente al hechicero del juego original) y posee un arsenal de cinco habilidades especiales. Cada habilidad es extremadamente poderosa, pero solo puede utilizarse una vez por partida:

- Teletransporte: mueve inmediatamente a un animal aliado a otra parcela válida.
- Curar: restaura la totalidad de la salud de un animal aliado.
- Intercambio: permuta la posición de dos animales seleccionados.
- Revivir: devuelve a la vida a un animal caído en combate, ubicándolo en una casilla adyacente al Granjero.
- Atrapar: lanza un lazo para atrapar a un animal rival en su casilla actual, inhabilitando su capacidad de movimiento durante varios ciclos del tablero.

3. ARQUITECTURA DEL SOFTWARE (POO)

3.1. Desacople entre lógica y motor gráfico

Uno de los pilares fundamentales en el desarrollo de Rancho ha sido la separación entre la actualización de los datos lógicos y su renderizado gráfico. Al principio del proyecto, las clases del juego como Animal, Tablero o Arena tenían métodos propios para dibujarse en pantalla. Esto generaba un fuerte acoplamiento con las bibliotecas gráficas externas, obligando a que la lógica de negocio dependiera de directivas de bajo nivel de OpenGL, freeglut y ETSIDilib.

Para lograr una arquitectura modular y robusta, se procedió a refactorizar el código centralizando la totalidad del procesamiento visual en una única clase especializada: el Renderizador. Este desacople se articula a través de los siguientes principios.

3.1.1. Limpieza del Modelo

Se eliminaron por completo los métodos `dibuja()` y las inclusiones de bibliotecas gráficas en las clases lógicas. Objetos como Animal o Menu pasaron a ser contenedores puros de estado (posiciones, vida, temporizadores), ignorando qué representación visual (sprite o modelo) les corresponde en pantalla. De este modo, la lógica del juego podría ser ejecutada teóricamente en una consola de texto sin requerir modificaciones en su código fuente.

3.1.2. Encapsulación y const-correctness

Para garantizar que el motor gráfico actúe exclusivamente como una "Vista" observadora y no altere accidentalmente la lógica del juego, la comunicación se realiza mediante punteros constantes. Las clases exponen su estado a través de métodos getters etiquetados con la palabra reservada `const`.

Si el Renderizador necesita pintar la el tablero, recibe un objeto inmutable, asegurando la integridad de los datos:

Fragmento 1. Ejemplo.

```
void Renderizador::dibujar(const Tablero* tablero) const {  
    // DIBUJAR FONDO Y LETRERO DE TURNOS  
    dibujarSprite("../assets/Sprites/tablero/tableroFondo.png", 512, 512, 480  
    / 2, 270 / 2, -1);  
  
    for (int i = 0; i < Constantes::FILAS_TABLERO; i++) {  
        for (int j = 0; j < Constantes::COLUMNAS_TABLERO; j++) {  
            const Animal* animal = tablero->getAnimalEnCasilla(i, j);  
            if (animal != nullptr) {  
                this->dibujar(animal);  
            }  
        }  
    }  
}
```

Máquina de Estados y Patrón Comando en la clase coordinadora, Juego.

Gestión de memoria y recursos (RAII). Patrones GRASP.
Regla de los Tres/Cinco/Cero (falta decidir cual, creo que Cero es la mejor).
Principios SOLID.

3.2. Estructura de clases

Abstracción, Encapsulamiento, Herencia y Polimorfismo.

El coordinador (Juego) y la gestión de fases (Menú, Tablero, Batalla).

Jerarquía polimórfica: clase abstracta Animal y sus hijas (Gallina, Cerdo, etc.), usando métodos virtuales puros y enlace dinámico.

Estructuras de control: Tablero (matriz de punteros) y Arena (fase de acción).

4. LÓGICA DE JUEGO

4.1. Fase estratégica (Tablero)

Validación de movimientos, oscilación de casillas y gestión de turnos.

4.1.1. Validación de movimientos

El sistema de movimiento en el tablero se ha diseñado bajo el patrón de separación de responsabilidades, desacoplando estrictamente la representación gráfica interactiva de la

lógica de reglas del juego. Durante su turno, el jugador puede desplazar el cursor libremente y mantener una pieza en el aire, pero el motor central del tablero evalúa el movimiento basándose exclusivamente en dos parámetros discretos: la casilla de origen y la casilla de destino.

Para gestionar esta transferencia de estado, dentro de un archivo *estructuras.h* se han definido estructuras de datos dedicadas que representan las coordenadas lógicas en la matriz del tablero.

Fragmento 2. Estructuras base para la gestión posicional.

```
struct Casilla {  
    int fila;  
    int columna;  
};  
  
struct Movimiento {  
    Casilla origen;  
    Casilla destino;  
};
```

Para maximizar la reutilización de código y evitar la redundancia (Principio DRY), el cálculo del rango de celdas alcanzables se gobierna de forma centralizada en la clase base *Animal*. Dado que la geometría de expansión radial alrededor de la pieza es idéntica para las diferentes piezas del juego, el método *movimientosPosibles()* no requiere ser redefinido individualmente en cada subclase.

En su lugar, el método de la clase madre pregunta de manera dinámica al atributo encapsulado *max_casillas_movidas_*. Este valor es configurado de manera específica en el constructor de cada animal dependiente (por ejemplo, asignando un alcance de 3 para la gallina y de 4 para la oveja). De este modo, la clase base resuelve la cuadrícula teórica de destinos válidos para cualquier pieza del rancho, reduciendo el coste de mantenimiento del software y facilitando la escalabilidad ante la futura incorporación de nuevos animales.

Fragmento 3. Cálculo polimórfico del alcance en la clase Animal.

```
std::vector<Movimiento> Animal::movimientosPosibles() const {  
    std::vector<Movimiento> movimientos;  
    Casilla origen = { casillaInicial_[0], casillaInicial_[1] };  
    int alcance = max_casillas_movidas_;
```

```

        for (int f = -alcance; f <= alcance; f++) {
            for (int c = -alcance; c <= alcance; c++) {
                if (f == 0 && c == 0) continue;

                int nuevaFila = origen.fila + f;
                int nuevaCol = origen.columna + c;

                if (nuevaFila >= 0 && nuevaFila <
Constantes::FILAS_TABLERO &&
                nuevaCol >= 0 && nuevaCol <
Constantes::COLUMNAS_TABLERO) {

                    Movimiento m;
                    m.origen = origen;
                    m.destino = { nuevaFila, nuevaCol };
                    movimientos.push_back(m);
                }
            }
        }
    }
    return movimientos;
}

```

Cuando el usuario interactúa con el teclado para confirmar el posicionamiento de una pieza, la clase Tablero procesa la solicitud mediante el método `esMovimientoLegal(const Movimiento& m)`. El algoritmo descarta movimientos que incurran en fuego amigo, evaluando que la casilla de destino no contenga un puntero a un animal del mismo equipo, y comprueba si la coordenada seleccionada se encuentra registrada dentro del contenedor de movimientos permitidos generado por el objeto Animal.

Una vez ratificada la legalidad de la jugada, el método `mover(const Movimiento& m)` efectúa la transferencia en la matriz de punteros y actualiza las propiedades físicas de renderizado de la pieza para asentarla en su nueva posición. Durante este proceso, resulta necesario reiniciar los vectores de velocidad residuales del objeto para garantizar que la rutina de actualización gráfica mantenga el sprite perfectamente estático en el centro de la casilla lógica asignada.

Fragmento 4. Sincronización espacial.

```

void Tablero::mover(const Movimiento& m) {
    Animal* pieza = jugadores_[turno_actual_]->getPiezaSeleccionada();

    // 1. Actualización de la matriz lógica
    casillas_[m.destino.fila][m.destino.columna] = pieza;

    // 2. Corrección de simetría vertical y sincronización gráfica
    float nuevaPosX = 141.0f + 11.0f + (22.0f * m.destino.columna);
    float nuevaPosY = 36.0f + 11.0f + (22.0f * (8 - m.destino.fila));
}

```

```

pieza->setPosX(nuevaPosX);
pieza->setPosY(nuevaPosY);

// 3. Freno de la inercia visual
pieza->setVelX(0);
pieza->setVelY(0);
pieza->setEnMovimiento(false);
}

```

4.1.2. Oscilación entre casillas

Para dotar al juego de la profundidad estratégica característica del título original, se ha implementado un sistema dinámico de oscilación de casillas. El tablero está compuesto por zonas fijas (siempre claras u oscuras) y zonas oscilantes, cuyo estado cambia cíclicamente entre neutro, claro y oscuro en función de los turnos transcurridos.

Desde el punto de vista del diseño orientado a objetos, toda esta lógica se ha encapsulado estrictamente dentro de la clase Tablero. Se utiliza un mapa de caracteres privado para almacenar la topología base del nivel, exponiendo el estado dinámico de cada posición únicamente a través de métodos de consulta (getters) constantes. De este modo, se asegura que componentes externos, como el motor de renderizado gráfico, puedan leer la información para dibujar la interfaz sin riesgo de alterar el estado lógico del juego.

```

int Tablero::getColorActualCasilla(int f, int c) const
{
    char tipo = tipo_casilla_[f][c];
    if (tipo == 'C') return 0; // Clara
    if (tipo == 'O') return 1; // Oscura
    if (tipo == 'P') return 3; // Poder

    if (tipo == 'N') {
        // Un turno completo equivale a 1 jugada del J1 + 1 jugada del J2.
        int ciclo = (turnos_totales_ / 2) % 4; // Fases: 0, 1, 2, 3

        if (ciclo == 0 || ciclo == 2) return 2; // Fase 0 y 2: Neutro
        if (ciclo == 1) return 0; // Fase 1: Claro
        if (ciclo == 3) return 1; // Fase 3: Oscuro
    }
    return 2; // Por defecto
}

```

El estado de la casilla donde se inicia una batalla tiene un impacto directo en la fase de combate. Si una unidad entra a la arena desde una casilla del color de su bando, recibe una bonificación equivalente al 50% de su salud máxima. Para gestionar esta mecánica de forma robusta, la clase base Animal se ha diseñado separando conceptualmente la

vida_base_ (salud permanente) del bonus_vida_casilla_ (escudo temporal). Todos los personajes heredan este comportamiento, garantizando que el tratamiento de las estadísticas sea uniforme gracias al polimorfismo, independientemente de si el combatiente es una Cabra, una Oveja o el Granjero.

Este incremento de salud funciona mecánicamente como un escudo de absorción de daño. Al finalizar un combate, el sistema evalúa los impactos recibidos frente a la bonificación de la casilla. Si el daño total es inferior o igual al bono táctico, la unidad se considera intacta y regresa al tablero general manteniendo su salud base inalterada. Únicamente el daño que logra quebrar y superar este escudo repercute en heridas permanentes para el animal.

```
// vida en batalla = vida base + bonus por estar en casilla de su color
int vidaAlEmpezarBatalla = animalGanador->getVidaBase() + animalGanador-
>getBonusVidaCasilla();
int danoRecibido = vidaAlEmpezarBatalla - animalGanador->getVida();
int bonus = animalGanador->getBonusVidaCasilla();

if (danoRecibido <= bonus) {
    // si el daño es menor al bonus, el bonus absorbió el ataque y vuelve con
    toda su vida base
    animalGanador->setVida(animalGanador->getVidaBase());
}
else {
    // si sobrepasa el bonus, el daño afecta a la vida
    int danoReal = danoRecibido - bonus;
    animalGanador->setVidaBase(animalGanador->getVidaBase() - danoReal);
    animalGanador->setVida(animalGanador->getVidaBase());
}

animalGanador->setBonusVidaCasilla(0); // limpia el bonus
```

Este enfoque modular no solo facilita la lectura y mantenimiento del código, sino que previene la corrupción de datos, ya que el cálculo del daño real y la actualización de las barras de salud se resuelven mediante el uso de setters controlados propios de la jerarquía de la clase Animal.

4.1.3. Habilidades especiales

El Granjero actúa como la pieza central de cada equipo y tiene un arsenal de habilidades especiales equivalentes a los hechizos del juego original. Para integrar estas acciones sin romper el flujo estratégico por turnos del tablero ni acoplar en exceso las clases, se ha diseñado e implementado una Máquina de Estados finitos dentro de la clase Tablero. El

sistema define un enumerado EstadoHechizo que permite al motor del juego pasar desde un estado inactivo hacia diferentes fases de selección de objetivos (aliados, enemigos o casillas vacías), dependiendo de la habilidad invocada mediante el teclado.

Fragmento 5. Máquina de estados para el control de habilidades.

```
enum EstadoHechizo
{
    INACTIVO,
    TELETRANSPORTE_SELECCIONAR_ALIADO,
    TELETRANSPORTE_SELECCIONAR_DESTINO,
    CURAR_SELECCIONAR_ALIADO,
    INTERCAMBIO_SELECCIONAR_ALIADO,
    INTERCAMBIO_SELECCIONAR_ENEMIGO,
    ATRAPAR_SELECCIONAR_ENEMIGO
};
```

Al activarse una habilidad, el tablero intercepta los comandos lógicos habituales. En lugar de permitir el desplazamiento de la pieza agarrada, el sistema evalúa la interacción del cursor según el estado activo a través del método ejecutarPasoHechizo(). Para asegurar una experiencia de usuario clara y evitar interferencias visuales, el renderizado gráfico de las casillas de movimiento posibles se supedita estrictamente a que la máquina se encuentre en estado INACTIVO.

Respecto al balance táctico del juego, la normativa dice que cada habilidad es extremadamente poderosa y solo puede ser utilizada una vez por partida. Para gestionar y restringir este recurso limitante, se ha incorporado una estructura de datos matricial que almacena de forma persistente la disponibilidad de las habilidades para ambos bandos:

Fragmento 6. Matriz de disponibilidad de un solo uso.

```
bool hechizoDisponible_[2][5];
```

Esta validación se comprueba al inicio de la rutina procesarTeclaHechizo(). Si el hechizo se completa con éxito (por ejemplo, al sanar un aliado o permutar posiciones), la posición correspondiente en la matriz cambia a false, consumiendo el hechizo.

Finalmente, para las habilidades que introducen efectos de estado prolongados, como es el caso de Atrapar (que inmoviliza a una unidad rival impidiéndole el movimiento), se ha encapsulado la lógica de control temporal en el método avanzarTurnosAtrapados(). Para garantizar un castigo íntegro y evitar un descuento de turnos prematuro, este método

se invoca sistemáticamente tras procesar cualquier jugada legal, justo antes de transferir el turno lógico al jugador contrario. Esto asegura la sincronización entre las restricciones de las piezas y el ciclo del tablero.

4.2. Fase de acción (Arena)

Algoritmos de colisión (AABB), confinamiento en los límites de la pantalla y gestión dinámica de proyectiles/ataques.

En esta recreación del Archon se buscó añadir mayor aleatoriedad en la jugabilidad del escenario arena, para ello se tomó la decisión de crear una función que colocaría las barreras de este escenario de juego de manera aleatoria en cada combate.

Para ello se decidió primero, dividir la zona de combate en 8 zonas diferentes, para que cada una de estas zonas esté ocupada por una de las ocho barreras que incluye nuestra arena. Para lograr esto se usaron las constantes `ZONA_DE_COMBATE_X` e `Y`, las cuales indican cual es el límite de las zonas de arena, y una variable interna de la función, `margen`, la cual evita que las barreras se coloquen en los bordes de los límites de la zona de pelea. Si se presta atención a la lógica de la función, se ve, que las variables `zona_x` y `zona_y` indican en cuantas filas y columnas se ha dividido la zona de combate, en este caso se ha dividido la zona de combate en 2 filas (`zona_x`) y 4 columnas (`zona_y`).

Una vez ya dividida las zona de la arena en mini zonas, se calcula que mini zona va a ocupar cada barrera, siendo así que, si se sigue la lógica del bucle de la función, y sabiendo que `NUM_DE_BARRERAS = 8`, la barrera 0 ocupa la columna 0 y fila 0 del área dividida de la arena, la barrera 1 ocupa la columna 1 y fila 0 de la zona dividida, y así sucesivamente.

Una vez ya calculado que barrera va a ocupar cada mini zona, se calculan los puntos de origen de posición de las barreras dentro de las mini zonas, que serán las esquinas de sus mini zonas respectivas, para más tarde determinar cual va a ser la posición final de cada barrera dentro de su mini zona. Para ello se suma al punto de origen de cada barrera un valor aleatorio contenido en su zona de ocupación (multiplicado por 0.7 para evitar que barreras vecinas se posicionen muy juntas). Una vez calculada la posición

final, se obtiene que las 8 barreras de arena se posicionan de formas diferentes en cada combate dentro del área de juego de este escenario

Fragmento 7. Ejemplo.

```
void Arena::colocarBarrerasAleatorias()
{
    float margen = 15.0f;
    float zona_x = (ZONA_DE_COMBATE_X - margen * 2) / 4.0f;
    float zona_y = (ZONA_DE_COMBATE_Y - margen * 2) / 2.0f;

    for (int i = 0; i < NUM_DE_BARRERAS; i++) {
        int columna = i % 4;
        int fila = i / 4;

        // esquina de cada zona dentro del cuadrado azul
        float origen_x = ARENA_MARGEN_X + margen + (columna *
zona_x);

        float origen_y = ARENA_MARGEN_Y + margen + (fila * zona_y);

        // posicion aleatoria dentro de esa zona
        barrera_x[i] = origen_x + (rand() % (int)(zona_x * 0.7f));
        barrera_y[i] = origen_y + (rand() % (int)(zona_y * 0.7f));
    }
}
```

Ya sabiendo como se posiciona dentro de la arena cada barrera, se va a proceder a explicar cómo funciona cada uno de los ataques que pueden producirse dentro del combate.

Para gestionar el funcionamiento de cada ataque se ha creado cinco ficheros tipo.h los cuales han sido denominados como ataque.h, dispar.h, embestida.h, golpe.h y onda.h. Los cuales corresponden a la lógica de los ataques en general, los ataques a distancia, los ataques que funcionan como una embestida, los que son cuerpo a cuerpo, y los que actúan en área respectivamente.

ataque.h:

```
#pragma once
#include <cmath>

// El dibujo del ataque lo hace renderizador.cpp usando getSprite(), getX(),
getY(), etc.

class Ataque {
protected:
    int        dano_;
    float      alcance_;
    float      recarga_;
    const char* sprite_;
    float      tamano_;
```

```

float        duracion_visual_;

float x_ = 0, y_ = 0;
float dirX_ = 0, dirY_ = 0;
bool  activo_ = false;
float tiempoActivo_ = 0;

public:
    Ataque(int dano, float alcance, float recarga, const char* sprite, float
tamanio, float duracion_visual)
        : dano_(dano), alcance_(alcance), recarga_(recarga),
          sprite_(sprite), tamanio_(tamanio), duracion_visual_(duracion_visual) {}

    virtual ~Ataque() {}

    int        getDano()      const { return dano_; }
    float      getAlcance()   const { return alcance_; }
    float      getRecarga()   const { return recarga_; }
    const char* getSprite()   const { return sprite_; }
    float      getTamanio()   const { return tamanio_; }
    float      getX()         const { return x_; }
    float      getY()         const { return y_; }
    bool       isActive()    const { return activo_; }

    virtual void activar(float x, float y, float dirX, float dirY) {
        x_ = x; y_ = y; dirX_ = dirX; dirY_ = dirY;
        activo_ = true; tiempoActivo_ = 0;
    }

    void desactivar() { activo_ = false; tiempoActivo_ = 0; }

    virtual void mover(float dt) = 0;
};

```

Este documento es el encargado de recibir cuanto daño hacen los ataques de cada animal, es decir, desde la clase de un animal, por ejemplo, granjero, le pasamos a esta clase los datos del ataque de esta figura, ya sea su daño, cuanto dura en pantalla, cuanto tarda en volver a estar disponible para atacar de nuevo etc. En esta clase también existen múltiples funciones las cuales sirven para que otras clases, como arena.cpp, puedan tener acceso a los datos del ataque de cierto animal, puesto que los datos de ataque son del tipo protected, para que ningún otro pueda modificarlos durante la ejecución del videojuego.

Esta clase también dispone de funciones las cuales reciben en que posición se encuentra el animal que posee el tipo de ataques que se trata, para así poder activar el ataque en la posición que ocupa el atacante dentro de la pantalla.

Además de todo lo ya mencionado, de manera interna, la clase gestiona cuanto tiempo el ataque está activo y cuando hay que actualizar este ataque.

disparo.h

```

#pragma once
#include "ataque.h"

class Disparo : public Ataque {

    float velocidad_ = 200.0f; // píxeles por segundo

public:
    Disparo(int dano, float alcance, float recarga, const char* sprite, float
tamanio, float duracion)
        : Ataque(dano, alcance, recarga, sprite, tamanio, duracion) {}

    void mover(float dt) override {
        if (!activo_) return;

        // Se desplaza en la dirección activada
        x_ += dirX_ * velocidad_ * (dt / 1000.0f);
        y_ += dirY_ * velocidad_ * (dt / 1000.0f);

        tiempoActivo_ += dt / 1000.0f;

        // Se desactiva al llegar al alcance máximo
        if (tiempoActivo_ >= alcance_ / velocidad_)
            desactivar();
    }
};

```

En esta clase se indica a que velocidad se mueven los proyectiles, se le pasa los datos del ataque de disparo del animal a la clase ataque, y se hace que la dirección de los disparos sea la misma que la dirección a la que se mueve el animal que dispara.

embestida.h:

```

#pragma once
#include "ataque.h"

// este ataque no es un proyectil, es mas bien un estado del animal
// Arena la lee con isEmbestida() para saber que debe mover al animal
// rapidamente.
// El daño se aplica por contacto en confirmarImpacto cuando el animal está
// embistiendo.

class Embestida : public Ataque {

    float velocidad_embestida_ = 300.0f; // px/seg durante la embestida
    float duracion_embestida_; // segundos que dura el impulso

public:
    Embestida(int dano, float duracion, float recarga, const char* sprite, float
tamanio)
        : Ataque(dano, 0.0f, recarga, sprite, tamanio, duracion),
          duracion_embestida_(duracion) {}

    // Devuelve la velocidad del impulso para que arena la aplique al animal
    float getVelocidadEmbestida() const { return velocidad_embestida_; }
    float getDirX() const { return dirX_; }
    float getDirY() const { return dirY_; }

    // La embestida es un tipo especial - arena lo detecta con esto

```

```

    bool isEmbestida() const { return true; }

    void mover(float dt) override {
        if (!activo_) return;
        tiempoActivo_ += dt / 1000.0f;
        if (tiempoActivo_ >= duracion_embestida_)
            desactivar();
    }
};

```

Esta clase es parecida a disparo, pero como indican los comentarios introducidos en este código, la embestida es más como un estado del animal, en esta clase se indica como de rápido se mueve la embestida, y cuanto dura este estado de embestida, los datos del constructor embestida son enviados desde la clase del animal que posea este ataque, y posteriormente estos datos son enviados a la clase central ataque.

golpe.h:

```

#pragma once
#include "ataque.h"

class Golpe : public Ataque {
public:
    Golpe(int dano, float alcance, float recarga, const char* sprite, float
tamanio, float duracion)
        : Ataque(dano, alcance, recarga, sprite, tamanio, duracion) {}

    void mover(float dt) override {
        if (!activo_) return;

        tiempoActivo_ += dt / 1000.0f;

        // El golpe aparece y desaparece tras su duración
        if (tiempoActivo_ >= duracion_visual_)
            desactivar();
    }
}

```

Esta clase, hace lo mismo que las anteriores, solo que el tiempo que dura activo el ataque golpe se ha dividido entre 1000, pues dt era un valor el cual hacia que el tiempo activo de este ataque fuese muy rápido.

onda.h:

```

#pragma once
#include "ataque.h"

class Onda : public Ataque {
    float expansion_ = 50.0f; // píxeles por segundo que crece
    float tamanioInicial_;
}

```

```

public:
    Onda(int dano, float alcance, float recarga, const char* sprite, float
tamano, float duracion)
        : Ataque(dano, alcance, recarga, sprite, tamano, duracion),
          tamanoInicial_(tamano) {}

    void activar(float x, float y, float dirX, float dirY) override {
        Ataque::activar(x, y, dirX, dirY);
        tamano_ = tamanoInicial_; // resetear tamaño al activar
    }

    void mover(float dt) override {
        if (!activo_) return;

        // La onda crece con el tiempo
        tamano_ += expansion_ * (dt / 1000.0f);
        tiempoActivo_ += dt / 1000.0f;

        if (tiempoActivo_ >= duracion_visual_)
            desactivar();
    }
};

```

Esta clase realiza lo mismo que las demás, pasar los datos de la clase animal que posea este ataque al constructor local, y al general, en esta clase lo que se hace es tomar en cuenta la posición del atacante para que el compilador sepa donde se está produciendo el ataque.

Una vez explicado como se gestionan los diferentes ataques que existen dentro del escenario de combate, se va a proceder a explicar como se gestiona de manera interna las interacciones de los diferentes elementos que pueden estar presentes dentro de arena, ejemplo: un disparo del contrincante, ¿qué pasa si una animal está cerca del otro?, ¿qué ocurre si una animal está cerca de una barrera?, etc.

Para poder explicar este punto se va a tener que analizar los archivos relacionados con las interacciones del juego

Por un lado, interacción.h

```

#pragma once
#include "ataque.h"
#include "animal.h"
#include "embestida.h"

class Interaccion {
public:
    static bool hayColision(Ataque* ataque, const Animal* rival);
    static bool procesarImpacto(Ataque* ataque, Animal* rival);
    static bool procesarEmbestida(Embestida* emb, Animal* atacante, Animal*
rival, float px_atac, float py_atac, float px_rival, float py_rival);

```

```

    static bool ataqueChocaBarrera(const Ataque* ataque, float bx, float
by, float semi_ancho = 10.0f, float semi_alto = 12.0f);

    static bool animalChocaBarrera(float x, float y, float bx, float
by, float semi_ancho = 10.0f, float semi_alto = 12.0f);

    static void mantenerDentroArena(float& x, float& y, float margen_x,
float margen_y, float zona_x, float zona_y, float radio = 11.0f);

    static bool ataqueEstaFuera(const Ataque* ataque, float lim_izq, float
lim_dch, float lim_arr, float lim_abj);
};

```

Como cabe esperar, dentro de este archivo se declaran las distintas funciones que van a ser desarrolladas a nivel lógico dentro de interacción.cpp.

El propio nombre de las funciones ya nos puede dar una idea de que va a gestionar cada una de ellas, por ejemplo, este archivo contiene las declaraciones de funciones las cuales van a comprobar si dentro de la arena ha habido alguna especie de impacto, o funciones las cuales obligan a los combatientes a mantenerse dentro de los límites de la arena, u otras que comprueban si un combatiente se está chocando con una barrera etc.

Para empezar se explicará la función hayColision():

```

bool Interaccion::hayColision(Ataque* ataque, const Animal* rival) {
    if (!ataque || !ataque->isActivo() || !rival) return false;
    float dx = ataque->getX() - rival->getPosX();
    float dy = ataque->getY() - rival->getPosY();
    return sqrt(dx * dx + dy * dy) < ataque->getTamano() / 2.0f + 11.0f;
}

```

La función hayColision comprueba si un ataque del animal atacante, ya sea jugador 1 o 2, ha impactado en el animal rival. Como se puede apreciar, primero se comprueba si los ataques no están desactivados, es decir, la función verifica si he producido un ataque o no por parte de uno de los dos animales, junto además de que comprueba de que el impacto no haya sido producido en el cuerpo de uno mismo; si no se ha producido un ataque, o bien se ha producido un ataque pero ha impactado en el combatiente que actúa a la ofensiva, y no en el combatiente rival, la función devolverá un valor false.

Si por lo contrario las condiciones del primer if de esta función no se cumplen, el código continuará compilación de esta función.

Para empezar se calculan cuales son las diferencias de distancia entre las posiciones x e y que ocupa el ataque del animal agresor dentro de la pantalla, y la posición dentro del espacio del animal rival, una vez obtenidos los valores de dx, y dy, se comprueba si estos valores han sido menores que el tamaño del ataque del animal atacante, si es así, la función le indicará al código que el ataque ha hecho impacto con el cuerpo del animal rival, devolviendo un true.

procesarImpacto():

```
bool Interaccion::procesarImpacto(Ataque* ataque, Animal* rival) {
    if (!hayColision(ataque, rival)) return false;
    rival->recibirDano(ataque->getDano());
    ataque->desactivar();
    return true;
}
```

Esta segunda función es la encargada de restarle el mismo número de puntos de vida que puntos de daño posee el ataque del atacante al animal rival, si el ataque del animal que ataca ha tenido éxito en impactar con el cuerpo del animal rival.

procesarEmbestida():

```
bool Interaccion::procesarEmbestida(Embestida* emb, Animal* atacante, Animal* rival,
    float px_atac, float py_atac,
    float px_rival, float py_rival)
{
    if (!emb || !emb->isActivo() || !rival) return false;
    float dx = px_atac - px_rival;
    float dy = py_atac - py_rival;
    if (sqrt(dx * dx + dy * dy) < 22.0f) {
        rival->recibirDano(emb->getDano());
        emb->desactivar();
        return true;
    }
    return false;
}
```

Esta función procesa algo parecido a lo que procesa función anteriormente explicada, solo que, a diferencia de procesarImpacto(), esta función lo que hace es procesar si la embestida del animal atacante ha impactado en el área de contacto del animal contrario.

Como en las anteriores funciones, se comprueba si se ha producido un ataque por parte del atacante, si es así se continua la lógica de esta función, si no se devuelve un valor false.

En caso de que las condiciones del primer if no se hayan satisfecho, la función calcula las distancias que existen entre la embestida del atacante, y la posición del animal atacado. Una vez obtenidos estos datos, se comprueban si estos son más pequeños que 22 pixeles, si es así, como en procesarImpacto, los puntos de vida del animal rival se reducen tanto como puntos de vida quiten la embestida recibida por el animal atacante, y se devuelve true, para indicar que la embestida si ha impactado en el animal rival.

AtaqueChocaBarrera():

```
bool Interaccion::ataqueChocaBarrera(const Ataque* ataque, float bx, float by,
    float semi_ancho, float semi_alto)
{
    if (!ataque || !ataque->isActivo()) return false;
    float dx = ataque->getX() - bx;
    float dy = ataque->getY() - by;
    return (dx > -semi_ancho && dx < semi_ancho &&
        dy > -semi_alto && dy < semi_alto);
}
```

La siguiente función se encarga de mirar si el ataque de una animal ha impactado con una barrera. La lógica es sencilla, primero se calcula que distancia x, y que distancia y existe entre el ataque de un nimal, y la posiciones en el espacio de una barrera. Posteriormente, ya después de haber calculado y guardado en las variables dx, y dy, setas distancias, se comprueban si estas han sido menores que las dimensiones que ocupan las barreras dentro de la pantalla, si es así, la función indica que el ataque está colisionando con una barrera.

animalChocaBarrera():

```
bool Interaccion::animalChocaBarrera(float x, float y, float bx, float by,
    float semi_ancho, float semi_alto)
{
    float dx = x - bx;
    float dy = y - by;
    return (dx > -semi_ancho && dx < semi_ancho &&
        dy > -semi_alto && dy < semi_alto);
}
```

Está función hace lo mismo que la anterior, solo que en vez de comprobar si un ataque está chocando con una barrera, comprueba si un animal está chocándose con una barrera, la lógica es la misma, toma que distancia hay entre la barrera y la posición actual del

animal en el espacio, y posteriormente mira si está distancia es menor que el ancho y alto de la barrera, si es así, la función indica que el animal se está chocando con la barrera.

mantenerDentroArena():

```
void Interaccion::mantenerDentroArena(float& x, float& y,
    float margen_x, float margen_y,
    float zona_x, float zona_y,
    float radio)
{
    if (x < margen_x + radio) x = margen_x + radio;
    if (x > margen_x + zona_x - radio) x = margen_x + zona_x - radio;
    if (y < margen_y + radio) y = margen_y + radio;
    if (y > margen_y + zona_y - radio) y = margen_y + zona_y - radio;
}
```

Esta función es la encargada de mantener en el límite de la zona de combate a los combatientes, es sencilla, básicamente recibe cuales son los límites de la arena, y cuanto es el radio de ocupación de los combatientes dentro del escenario de combate, para posteriormente, con estos datos, obligar a los peleadores a mantenerse dentro de los límites establecidos en arena.h.

ataqueEstaFuera():

```
bool Interaccion::ataqueEstaFuera(const Ataque* ataque,
    float lim_izq, float lim_dch,
    float lim_arr, float lim_abj)
{
    if (!ataque || !ataque->isActivo()) return false;
    return (ataque->getX() < lim_izq || ataque->getX() > lim_dch ||
        ataque->getY() < lim_arr || ataque->getY() > lim_abj);
}
```

Y por último, esta función hace lo mismo que la anterior, solo que esta se encarga de que los ataques de los animales no se salgan del área de combate, esta función es muy útil, sobre todo con los ataques de tipo disparo u embestida, básicamente, comprueba que las posiciones en el espacio de los ataques no se localicen nunca por fuera de la zona límite de arena.

4.3. Las piezas (Animales)

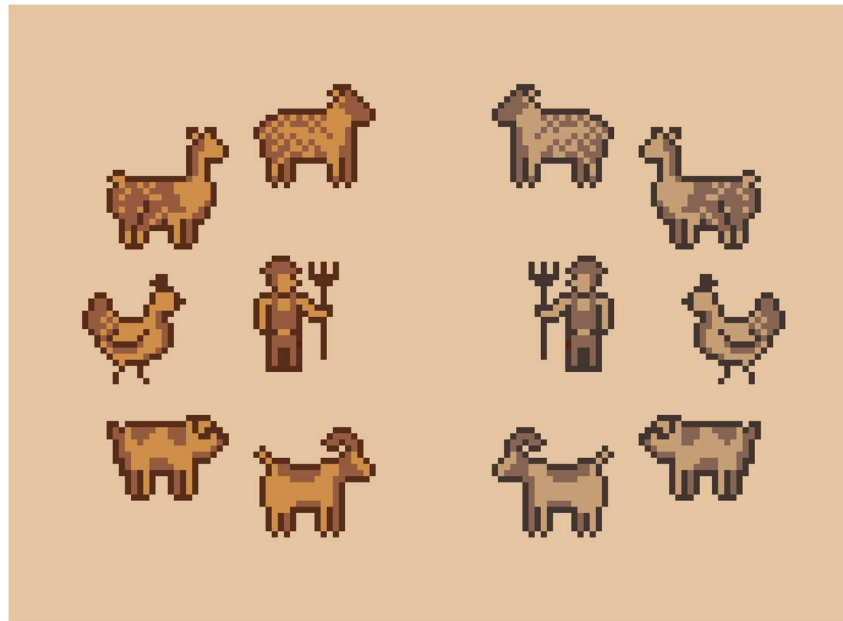


Figura 3. Granjero y animales de cada bando de Rancho.

Características de cada animal, tipo, vida... Foto del animal.

Para el granjero, habilidades equivalentes a los hechizos del Archon original.

En esta recreación del Archon se optó que en cada equipo de cada bando dispusiera de 9 gallinas, 2 cabras, 2 llamas, 2 cerdos, 2 ovejas y un granjero.

Gallina:



Si se usan términos relacionados con el ajedrez, la gallina dentro de este juego actúa como peón, esta unidad es la que más repite en cada equipo, y es la primera línea de figuras de cada bando.

Si se inspecciona su clase respectiva, es decir, gallina.h, se puede obtener más datos relacionados con esta unidad.

Fragmento 8. Clase Gallina.

```
#pragma once
#include "animal.h"
#include "disparo.h"
```

```
class Gallina : public Animal
{
public:
    Gallina(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 14;
        max_casillas_movidas_ = 2;
        nFrames = 8;
        especie_ = GALLINA;
        ataque_ = new Disparo(2, 140.0f, 0.8f,
        "../assets/Sprites/menu/selector.png", 15.0f, 0.35f);
    }

    const char* getTipoAtaque() const override { return "Disparo";}
};
```

Si se presta atención a las líneas incluidas dentro de esta clase, se puede apreciar que la unidad de la gallina dispone de 14 puntos de vida, y que dentro del escenario tablero, esta unidad puede moverse un máximo de 2 casillas, en la siguiente imagen se presenta de forma clara que movimientos dentro de tablero tiene permitido la gallina realizar:



Por otro lado, dentro de esta clase se puede observar también que tipo de ataque dispone esta unidad dentro del escenario arena, el ataque es uno de tipo disparo. Si se accede a la clase `disparo.h`, dentro de su constructor se localizan los siguientes parámetros.

```
Disparo(int dano, float alcance, float recarga, const char* sprite, float
tamano, float duracion)
```

Sabiendo los parámetros del constructor, se puede llegar a la conclusión que el ataque disparo de la gallina posee una capacidad de daño de 2 puntos de vida, un alcance de

hasta 140 píxeles de pantalla, con un tiempo de recarga de 0.8 segundos, tamaño 15 píxeles, y una duración de 0.35 segundos.

Cabra:



Por otro lado, la unidad cabra son las piezas localizadas en las esquinas de ambos equipos. Indagando más dentro de su clase, cabra.h, se deduce lo siguiente

```
#pragma once
#include "animal.h"
#include "embestida.h"

class Cabra : public Animal
{
public:
    Cabra(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 20;
        max_casillas_movidas_ = 4;
        nFrames = 5;
        especie_ = CABRA;
        ataque_ = new Embestida(8, 0.3f,
1.2f, "../assets/Sprites/creditos/botella.png", 20.0f);
    }

    const char* getTipoAtaque() const override { return "Embestida"; }

    //void actualizar(float dt) override;
};
```

La unidad cabra presenta 20 puntos de vida, y una capacidad de movimiento dentro del escenario tablero de 4 casillas desde su posición de origen.



Aunque dentro de la imagen no se aprecia, la cabra como las demás figuras presenta la capacidad de retroceder hasta 4 casillas hacia atrás desde su posición de origen. Siguiendo con el código de esta clase, se observa que el ataque de la cabra es una embestida, el constructor de embestida tiene esta forma:

```
Embestida(int dano, float duracion, float recarga, const char* sprite, float
tamanio)
```

Observando bien el constructor, el daño que produce esta embestida dentro de la clase cabra produce unos 8 puntos de daño, el movimiento dura 0.3 segundos, y tarda en recargarse 1.2 segundos. El tamaño de este ataque es de 20 pixeles

Llama:

La llama es la unidad más cercana a la unidad granjero.



Clase llama.h:

```
#pragma once
#include "animal.h"
#include "disparo.h"

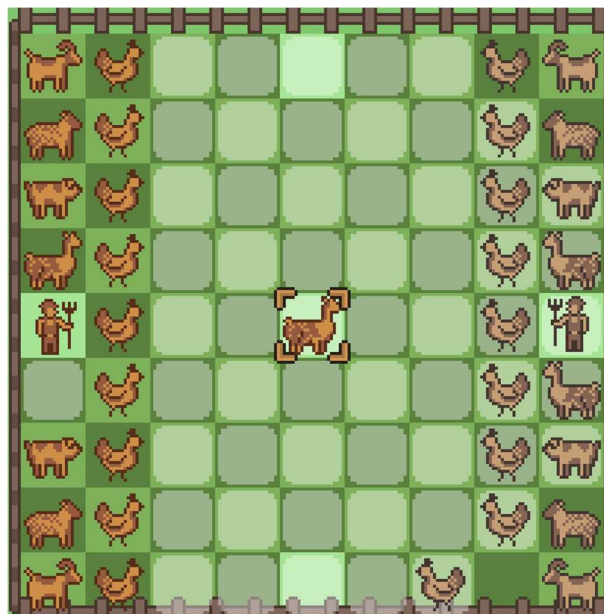
class Llama : public Animal
```

```

{
public:
    Llama(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 12;
        max_casillas_movidas_ = 6;
        nFrames = 5;
        especie_ = LLAMA;
        ataque_ = new Disparo(2, 140.0f, 0.8f,
            "../assets/Sprites/menu/selector.png", 15.0f, 0.35f);
    }
    const char* getTipoAtaque() const override { return "Disparo"; }
    //void actualizar(float dt) override;
};

```

Si se analiza con detenimiento el código de la clase llama, se puede observar que la llama posee 12 puntos de vida, una capacidad de movilidad de 6 casillas dentro del tablero, un ataque de mecánica disparo.



Con anterioridad ya hemos visto como es el constructor de disparo:

```

Disparo(int dano, float alcance, float recarga, const char* sprite, float
tamano, float duracion)

```

Si prestamos atención en los valores que toman cada uno de los parámetros del disparo de la llama, se puede apreciar que los valores del disparo de llama son similares a los valores del disparo de gallina.

Cerdo:



El cerdo es el único animal que presenta la peculiaridad de que su ataque es de estilo onda expansiva, es decir es el único animal cuyo ataque afecta a los rivales que se encuentren dentro del área de acción

El cerdo, dentro del tablero, ocupa una posición entremedias de las unidades oveja, y las unidades llama.

Cerdo.h:

```
#pragma once
#include "animal.h"
#include "onda.h"

class Cerdo : public Animal
{
public:
    Cerdo(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 10;
        max_casillas_movidas_ = 4;
        nFrames = 5;
        especie_ = CERDO;
        ataque_ = new Onda(3, 100.0f,
        1.0f, "../assets/Sprites/menu/selector.png", 20.0f, 0.6f);
    }
    const char* getTipoAtaque() const override { return "Onda"; }
    //void actualizar(float dt) override;
};
```

Dentro de su clase, se declara que el cerdo tiene 10 puntos de vida, y una capacidad de movimiento dentro del tablero similar a la cabra.



Como se había dicho con anterioridad, la unidad cerdo posee un ataque el cual puede dañar a los animales de alrededor, puesto que es un ataque tipo onda expansiva. El constructor de este ataque lleva declarados los siguientes parámetros

```
Onda(int dano, float alcance, float recarga, const char* sprite, float tamano, float duracion)
```

Por lo tanto, según los valores que se encuentran dentro de la clase cerdo, se establece que su ataque produce un daño de 3 puntos de vida, un alcance de 100 pixeles, un periodo de recarga de ataque de 1 segundo, un tamaño de ataque de área de 20 pixeles, y un tiempo de duración de ataque de 0.6 segundos.

Oveja



La oveja se trata de la unidad localizada entre las cabras y los cerdos dentro del escenario tablero, este animal presenta un ataque similar a la unidad cabra, un ataque de embestida.

Clase oveja.h:

```
#pragma once
#include "animal.h"
#include "embestida.h"

class Oveja : public Animal
{
public:
    Oveja(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 10;
        max_casillas_movidas_ = 3;
        nFrames = 5;
        especie_ = OVEJA;
        ataque_ = new Embestida(5, 0.2f,
0.9f, "../assets/Sprites/creditos/botella.png", 16.0f);
    }
    const char* getTipoAtaque() const override { return "Embestida"; }
    //void actualizar(float dt) override;
};
```

Como se ve en las líneas de la clase oveja, esta unidad dispone de 10 puntos de vida y de una movilidad de 3 casillas dentro del tablero.



La embestida de la oveja inflige menos daño de la que inflige la embestida de la cabra, tres puntos de daño menos para ser exactos, sin embargo el tiempo de recarga de la embestida de la oveja es menor que el de la cabra, aunque su tamaño en pantalla y duración también es menor que el de la cabra

Granjero



Es la pieza central de cada bando, esta unidad es fundamental en ambos equipos, pues la única que puede realizar hechizos, junto además de que es la figura que mayor movilidad presenta dentro del tablero. Esta unidad también presenta la peculiaridad de ser la única que ataca cuerpo a cuerpo

Granjero.h

Como se ha hecho con el resto de figuras, se va a analizar el código de su archivo.h respectivo

Según indican sus líneas de código, el granjero posee 10 puntos de vida dentro del escenario de combate, junto a una movilidad de 10 casillas dentro del tablero.

Como se ha dicho con anterioridad, el ataque del granjero es un ataque estilo cuerpo a cuerpo, el cual según su constructor

```
Golpe(int dano, float alcance, float recarga, const char* sprite, float tamano, float duracion)
```

Este ataque inflige un daño de 4 puntos, un alcance de 100 pixeles, una recarga de 0.6 segundos, un tamaño de pantalla de 16 pixeles y una duración de ataque de 0.4 segundos

Escribir lo que hacen los hechizos cuando se implementen dentro del develop

5. METODOLOGÍA Y CONTROL DE VERSIONES

Para garantizar el éxito del proyecto, la colaboración fluida entre los cinco miembros del equipo y la integridad del código se ha establecido una metodología de trabajo rigurosa basada en estándares profesionales de la industria del software.

5.1. Flujo de trabajo (Gitflow)

La gestión del control de versiones se ha articulado en torno a la metodología Gitflow, estructurando el repositorio en diferentes niveles de aislamiento para proteger el código funcional:

- Rama master: contiene únicamente las versiones estables y funcionales del juego (releases). Es el reflejo del producto final.
- Rama develop: actúa como la rama principal de integración. Todo el código nuevo se fusiona aquí para ser testeado en conjunto.
- Ramas feature/: todo nuevo desarrollo se aísla en su propia rama a partir de develop. Para organizar el trabajo en "Rancho", se han utilizado ramas temáticas específicas como feature/animales, feature/tablero, feature/arena, feature/juego, feature/teclado o feature/creditos.
- Ramas hotfix y release: reservadas para correcciones críticas de errores en producción y para la preparación de versiones estables, respectivamente.

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	yesterday			Default	
feature/juego	19 hours ago		0	9	#45
feature/clase-jugador	yesterday		25	2	
feature/animales	2 days ago		19	3	
master	2 months ago		78	0	

Figura 4. Ramas del repositorio en GitHub.

5.2. Integración y revisión de código

Para mantener la estabilidad de la rama develop, se ha prohibido la subida directa de código (commits directos). La integración de las ramas feature/ se realiza exclusivamente a través de Pull Requests (PR).

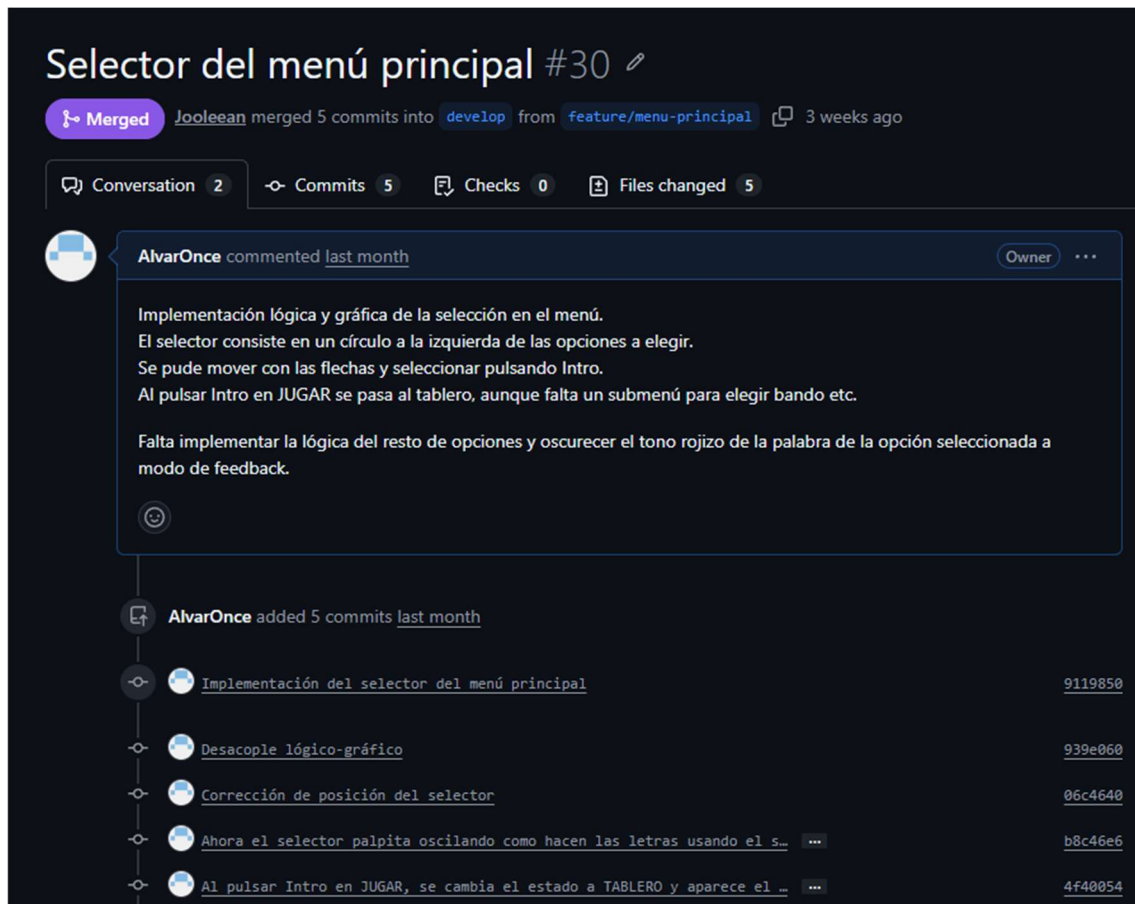


Figura 5. Pull Request para fusionar el menú principal a la rama develop.

Este sistema asegura que cualquier nueva funcionalidad implementada por un miembro del equipo sea revisada y validada antes de fusionarse. El uso de PRs permite la resolución temprana de conflictos de fusión (merge conflicts) y asegura que el código entrante cumple con la guía de estilo del proyecto.

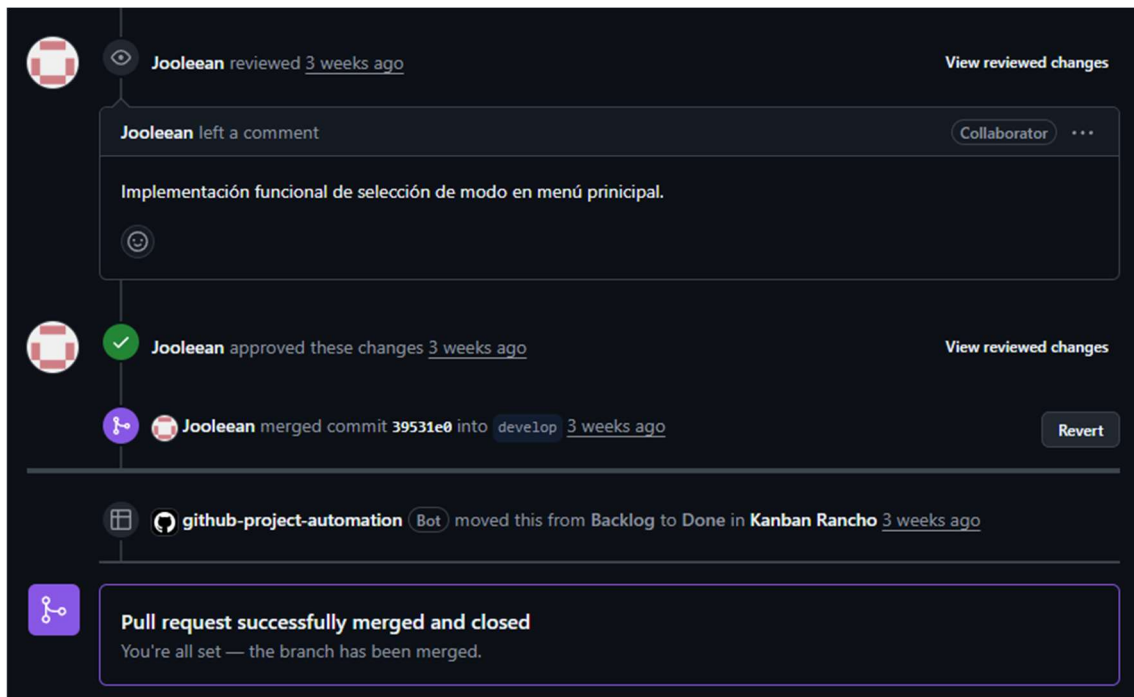
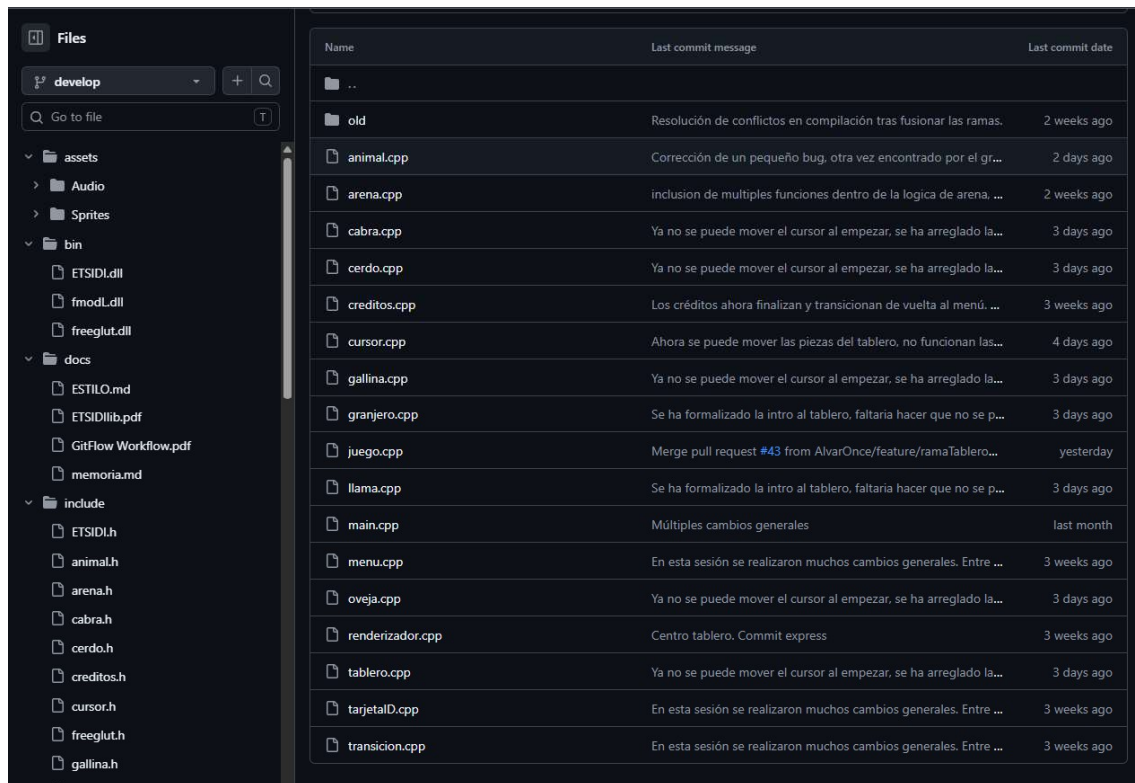


Figura 6. Cambios aprobados tras revisar los archivos cambiados.

5.3. Estructura del repositorio

El repositorio de GitHub se ha organizado siguiendo una estructura de directorios limpia y modular, separando el código fuente de los recursos y la documentación:

- assets/: Almacena los recursos multimedia del juego, subdividido a su vez en Audio (efectos y música) y Sprites (texturas e imágenes de los animales, tablero, etc.).
- bin/: Contiene las librerías dinámicas necesarias para la ejecución del programa en entornos Windows (ETSIDI.dll, freeglut.dll).
- docs/: Recopila la documentación técnica del proyecto, incluyendo manuales (ESTILO.md), instrucciones de librerías y la presente memoria.
- include/: Directorio reservado para los ficheros de cabecera (.h), donde se declaran las clases y estructuras (ej. animal.h, tablero.h, arena.h).
- src/: Contiene los ficheros de código fuente (.cpp) donde se definen e implementan los métodos de las clases (ej. animal.cpp, main.cpp, juego.cpp).



Name	Last commit message	Last commit date
...		
old	Resolución de conflictos en compilación tras fusionar las ramas.	2 weeks ago
animal.cpp	Corrección de un pequeño bug, otra vez encontrado por el gr...	2 days ago
arena.cpp	inclusion de multiples funciones dentro de la logica de arena. ...	2 weeks ago
cabra.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
cerdo.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
creditos.cpp	Los créditos ahora finalizan y transicionan de vuelta al menú. ...	3 weeks ago
cursor.cpp	Ahora se puede mover las piezas del tablero, no funcionan las...	4 days ago
gallina.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
granjero.cpp	Se ha formalizado la intro al tablero, faltaria hacer que no se p...	3 days ago
juego.cpp	Merge pull request #43 from AlvarOnce/feature/ramaTablero...	yesterday
llama.cpp	Se ha formalizado la intro al tablero, faltaria hacer que no se p...	3 days ago
main.cpp	Múltiples cambios generales	last month
menu.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago
oveja.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
renderizador.cpp	Centro tablero. Commit express	3 weeks ago
tablero.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
tarjetaID.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago
transicion.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago

Figura 7. Estructura del repositorio.

5.4. Guía de estilo y convenciones de código

Para garantizar la calidad, legibilidad y estabilidad del proyecto entre todos los desarrolladores, se ha redactado y aplicado una guía de estilo basada en gran parte en las convenciones de Google C++ Style Guide.

Fragmento 9. Ejemplo de estilo y sintaxis.

```
void Animal::animar(float dt) {
    timer = timer + dt;

    if (timer > msStep) {
        if (frameActualX_ < nFrames-1) frameActualX++;
        else frameActualX_ = 0;
        timer = timer - msStep;
    }
}
```

5.4.1. Nomenclatura

- Clases y estructuras (PascalCase): la primera letra de cada palabra en mayúscula (ej. class Tablero, struct CoordenadaArena).

- Métodos y funciones (`camelCase`): la primera letra en minúscula y las siguientes palabras inician en mayúscula (ej. `void actualizarEstado()`, `bool movimientoValidoTablero()`).
- Variables locales y parámetros (`snake_case`): todo en minúsculas, separando palabras con guiones bajos (ej. `int vida_actual`, `float posicion_x`).
- Atributos de clase (`snake_case_`): los miembros privados terminan con un guion bajo para diferenciarlos rápidamente de las variables locales dentro de los métodos (ej. `Tablero* tablero_juego_`, `int dano_base_`).
- Constantes y enums (`SCREAMING_SNAKE_CASE`): todo en mayúsculas con guiones bajos (ej. `MAX_ANIMALES`, `enum class EstadoJuego { MENU, TABLERO, BATALLA }`).
- Ramas del repositorio: emplean el formato `feature/kebab-case` (ej. `feature/habilidades-granjero`).

5.4.2. Formato y sintaxis

- Llaves `{ }`: Ambas llaves, la de apertura y cierre van en una nueva línea.
- Sangría: se emplean estrictamente 4 espacios por cada nivel de indentación.
- Punteros y Referencias: el asterisco (`*`) o el ampersand (`&`) se escriben pegados al tipo de dato, no a la variable (ej. `Animal* gallina`).
- Getters y Setters: el getter debe llamarse igual que el atributo (sin el guion bajo final y sin el prefijo `get`) y el setter debe empezar con el prefijo `set_` (ej. `vida()` y `set_vida()`).

5.4.3. Arquitectura y POO

- Protección de cabeceras: todo archivo `.h` debe comenzar con la directiva `#pragma once` para evitar inclusiones múltiples.
- Encapsulamiento: los atributos de clase son siempre `private` o `protected`, ubicándose por defecto al principio de la clase. Los métodos de acceso se implementan únicamente cuando es estrictamente necesario exponer o alterar un dato desde el exterior.

- Const-correctness: se etiqueta con la palabra reservada `const` a todos aquellos métodos observadores que consultan datos pero no alteran el estado interno del objeto (ej. `int setVida() const;`).
- Gestión de memoria: siguiendo el principio de asimetría, toda asignación dinámica de memoria realizada con `new` tiene asegurada su correspondiente liberación con `delete` en el destructor de la clase, garantizando el cumplimiento del patrón RAII y evitando fugas de memoria.

6. ELEMENTOS ORIGINALES Y OPCIONALES

6.1. Personalización temática

Elementos únicos de "Rancho" (gráficos, música, sonidos).

6.1.1. Gráficos y animaciones

Fragmento 10. Ejemplo.

```
void dibujarSprite (
    const char* rutaImagen, // ruta del sprite a dibujar
    float ancho, // ancho y alto del sprite, si es un sprite sheet, el ancho y
    alto total del sprite sheet, no del frame individual
    float alto,
    float posX, // posición del centro del sprite en la ventana, no de la
    esquina superior izquierda
    float posy,
    float capa, // la capa en la que se dibuja el sprite, cuanto mayor sea el
    número, más cerca del espectador estará el sprite (capa = profundidad)
    int rows = 1, // número de filas y columnas del sprite sheet, si el sprite
    no tiene animación, se deja en 1
    int cols = 1,
    int stateX = 0, // número de frame en la dirección horizontal (columna),
    empezando por 0
    int stateY = 0,
    bool transparencia = true // si el sprite tiene transparencia, se activa
    el alpha test para no dibujar los píxeles completamente transparentes, esto
    mejora rendimiento y evita artefactos raros entre animales
) const;
```

6.1.2. Música y efectos de sonido

6.2. Funcionalidades extra

Menú principal, créditos, guardar partida, pantalla final con estadísticas de la partida

7. CONCLUSIONES

Retos superados.

Trabajo futuro, posibles mejoras...